

C2SM Git for Advanced Workshop 2025

12 September 2025

Annika Lauber, Mikael Stellio

Outline

Part 0: Recap on Git

- Why use Git?
- Practical example
- Local Git workflow

Part 1: Examining a Git Repository

- Useful commands to examine Git repositories
- Exercises 1-2

Part 2: Git Workflow

- Web interface workflow
- Web interface demonstration
- Useful workflow commands
- Git cherry-pick
- Custom Git Hooks
- Exercises 3-7

Part 3: Nesting Git Repositories

- Using Git submodules
- Exercise 8

Part 4: Useful Tools and Resources

- External Git tools
- Git resources
- Git in VS Code demonstration

Schedule

09:00 – 09:10 Welcome & Git Recap

09:10 – 10:00 Exercise 1 – 2

10:00 – 11:00 Exercise 3 – 5

11:00 – 11:20 Coffee Break

11:20 – 11:40 Exercise 6 – 7

11:40 – 12:10 Exercise 8

12:10 – 12:30 Useful Tools demonstration

Part 0:

Git Recap

Why Use Git?

- Tracks file changes in a documented way

Without versioning

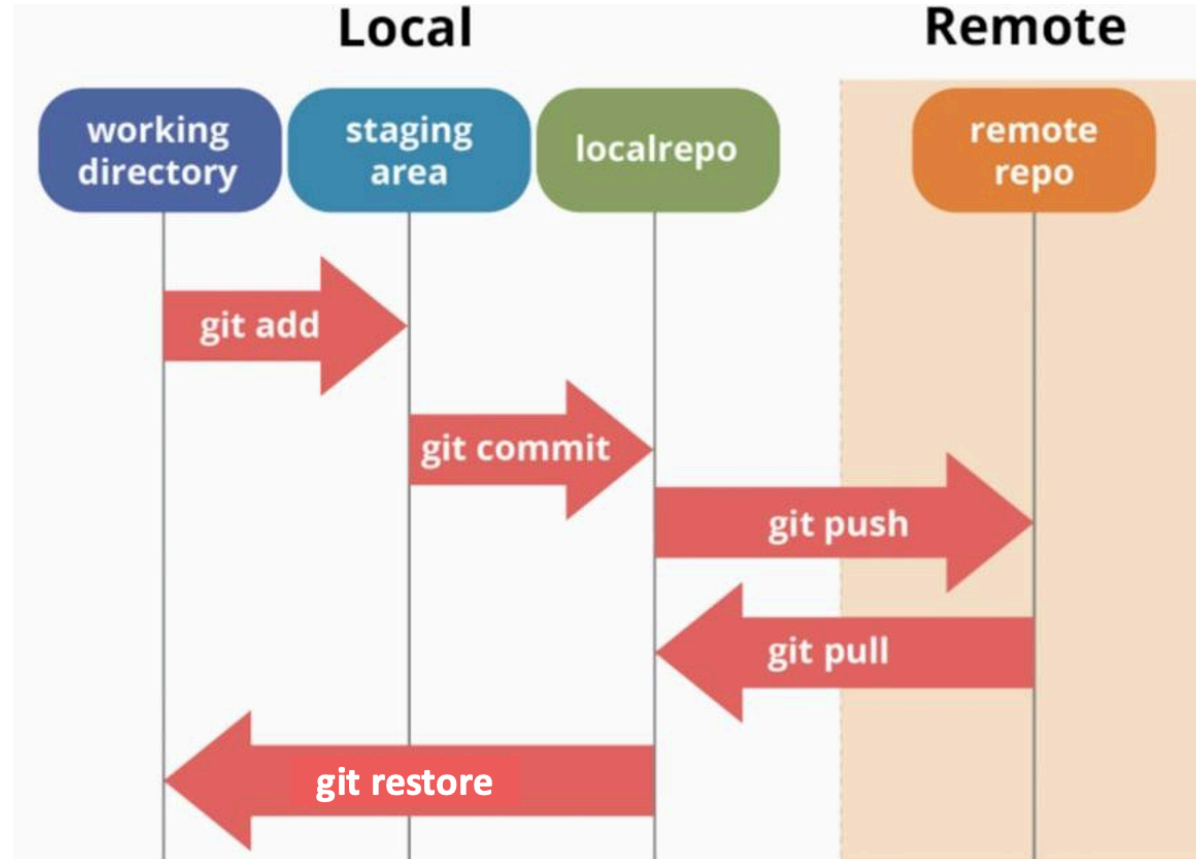
- conference_schedule_v0.txt
- conference_schedule_v1.txt
- conference_schedule_v2.txt
- conference_schedule_v2_dj.txt
- conference_schedule_v2_orch.txt
- conference_schedule_v3.txt

With versioning

- conference_schedule.txt
- .git (file, which contains information of all changes including reasons for changes, if well documented)

- Allows us to work simultaneously on the same code (when using remote server)
 - Alone (on different computers)
 - Multiple people in a collaboration
- Maintain several parallel versions of the same code in a systematic way.
- Many tools available (web-based services, graphical interfaces, etc.)

Local Workflow Recap



Source: <https://dev.to/mollynem/git-github--workflow-fundamentals-5496>

Part 1:

Examining a Git Repository

Useful Commands

`git log`

shows the commits in a repository

`git blame`

shows when what part of file was changed last by which commit

`git diff`

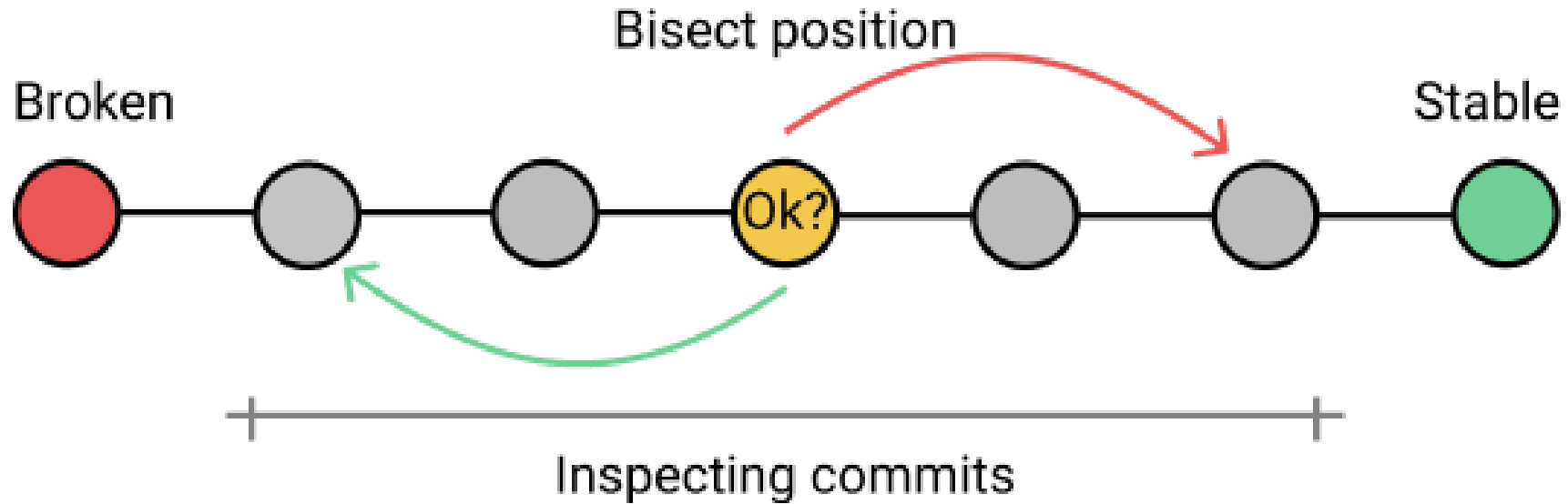
shows changes between commits, commit and working tree, etc.

`git show`

shows both commit information AND commit diff

These commands have many different options for customizing the output (explored in Exercise 1)

git bisect



- Iteratively locate the commit where a change occurred
- Requires a linear history to work correctly

Examining a Git Repository: Exercises

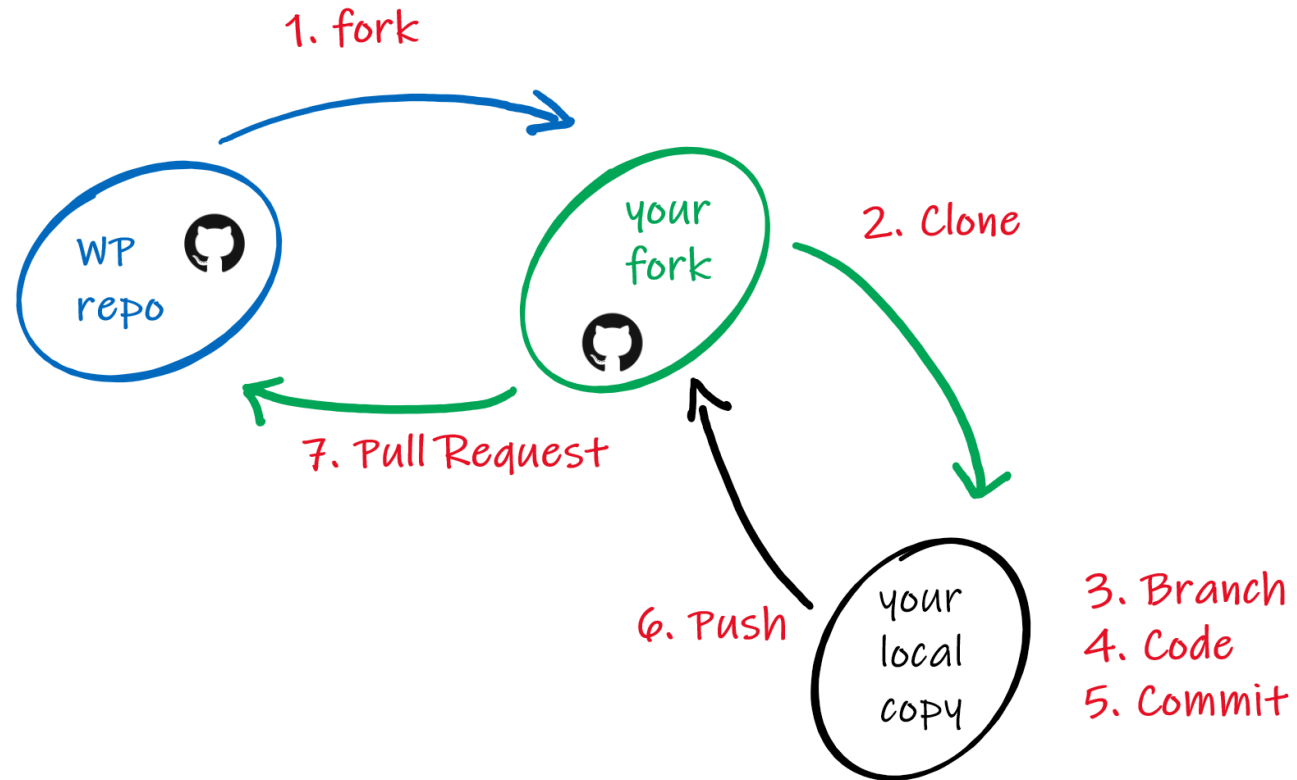
- Exercise 1: `git log`, `git blame`, `git diff`, and `git show`
- Exercise 2: `git bisect`

Exercises can be found at: <https://github.com/C2SM/git-course/tree/main/advanced>

Part 2:

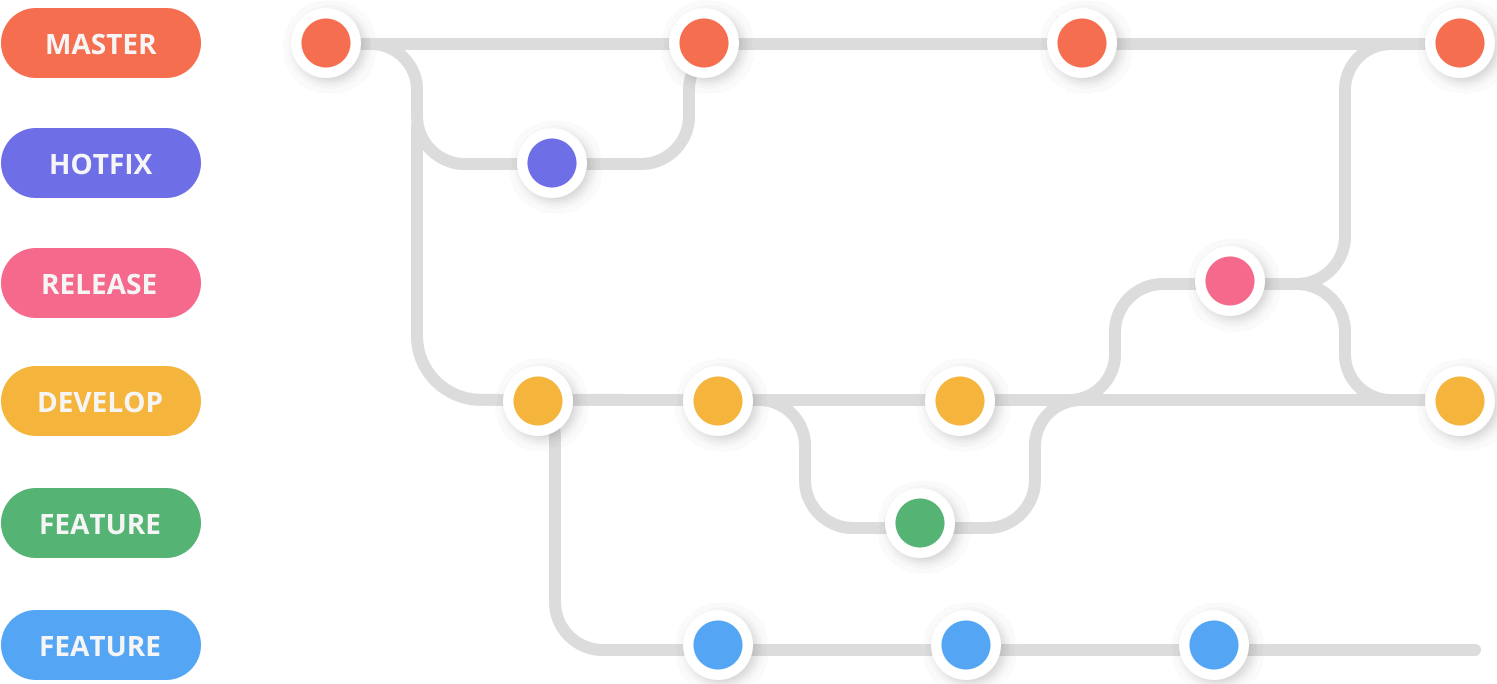
Git Workflow

Web Interface Workflow – Repository Level



Source: <https://developer.wordpress.org/block-editor/contributors/code/git-workflow/>

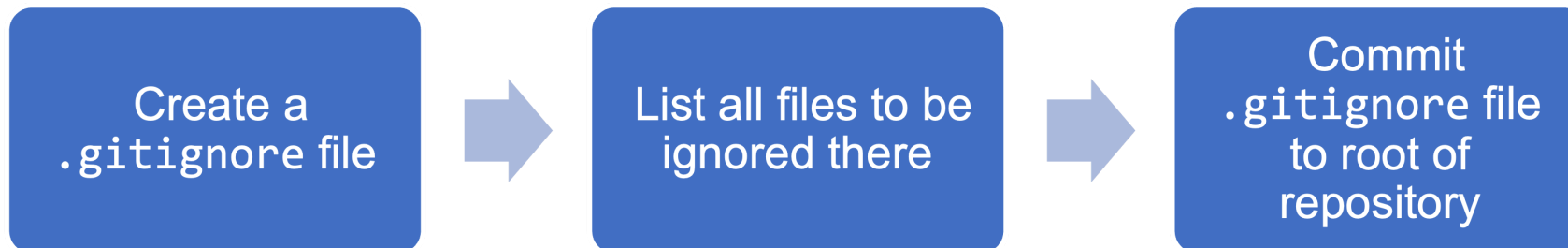
Web Interface Workflow - Branch Level



.gitignore

- Tell Git to disregard files you don't want committed.
- Best practice is to ignore binaries, intermediate files, files that can be generated from files in your repository, etc.

```
*~  
*.exe  
netcdf-*  
bin  
!bin/gen_info.sh
```



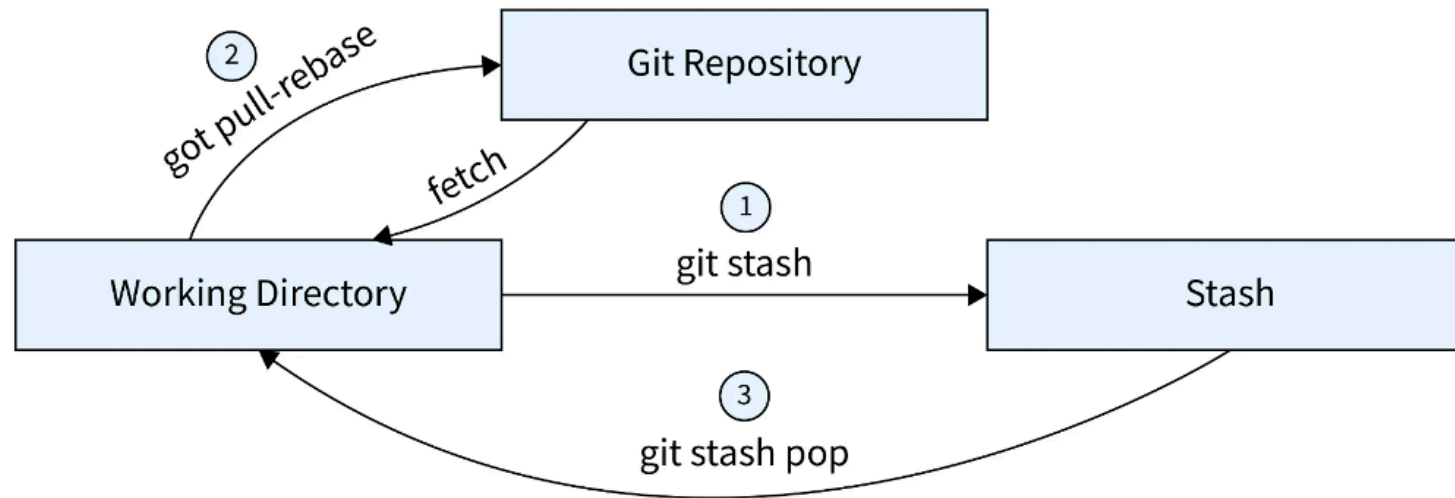
.gitkeep

- Git keeps track of files, not folders
- Put an empty `.gitkeep` file in any folder you would like to keep in the repository
- Commit the `.gitkeep` file to the Git repository

Note: This is a convention that has developed, not an official Git feature like `.gitignore`.

git stash

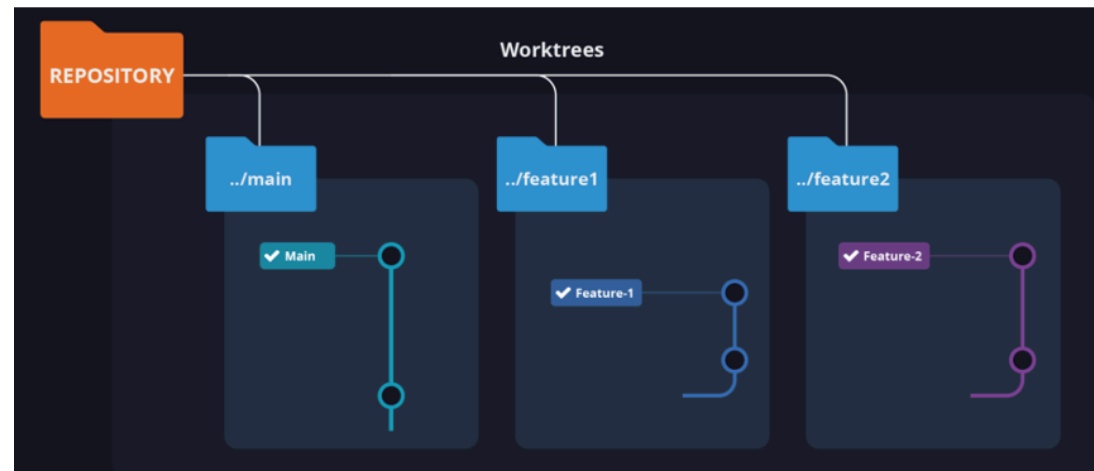
- Allows you to save bits of work without committing them and reuse them later
- Useful when:
 - you need to pull changes, but have uncommitted changes
 - you need to switch branch, but have uncommitted changes



Source: <https://www.scaler.com/topics/git/git-stash-pop/>

git worktree

- Can checkout and work with multiple branches of a repository with a single clone
- Worktrees share a single `.git` directory, which:
 - **saves memory and time** compared to multiple clones
 - keeps the git configuration **centralized**
- Can build/test multiple branches simultaneously



Source: <https://www.gitkraken.com/learn/git/git-worktree>

GitHub Web Interface Demonstration

<https://github.com/C2SM/git-workflow-practice>

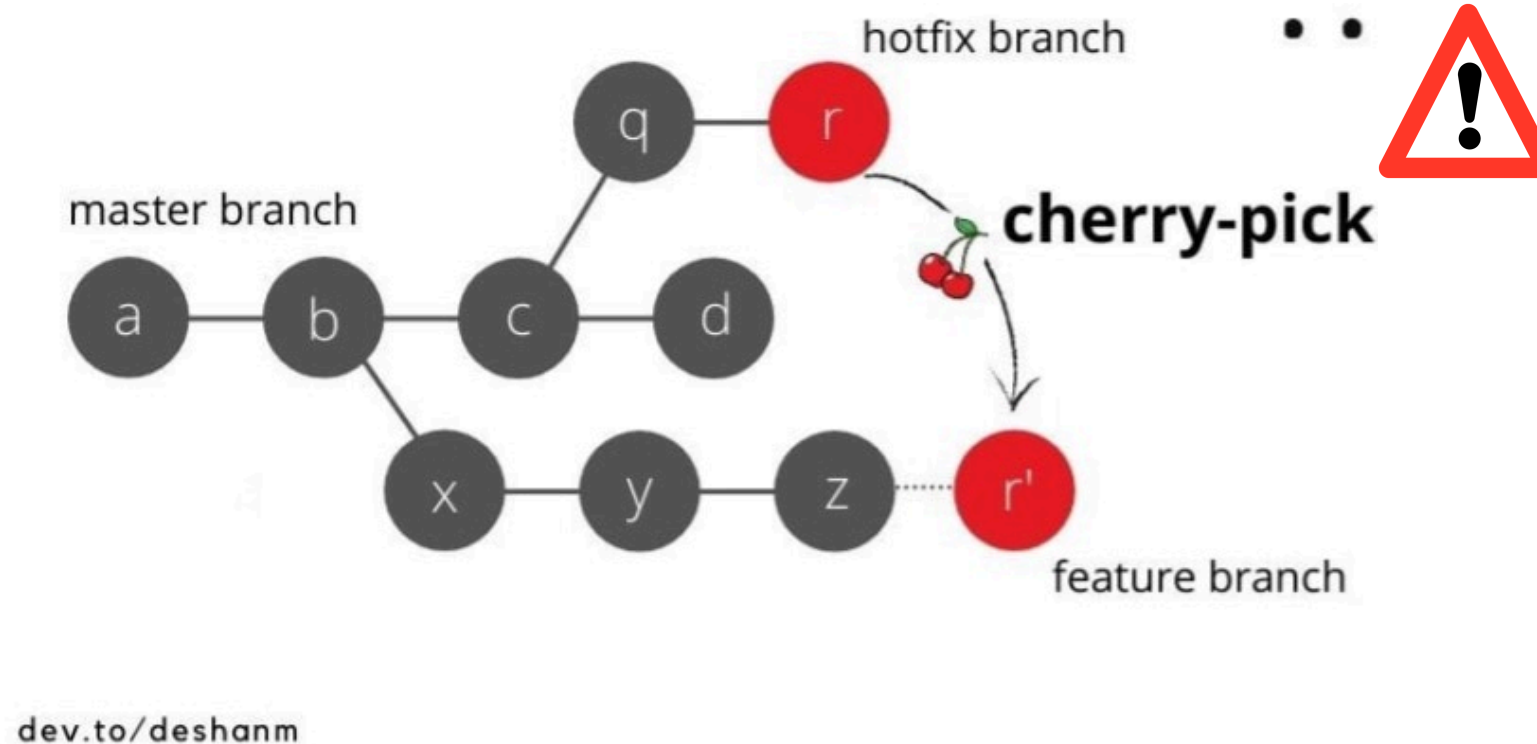
Git Workflow - Exercises

- Exercise 3: `.gitignore`
- Exercise 4: `git stash` and `git worktree`
- Exercise 5: practice the git workflow

Coffee Break



git cherry-pick: Snagging one Commit



- Grabs one commit and puts it at the head of another branch.
- Uses a different commit ID for the same file changes.

Custom Git Hooks

Custom Git Hooks

- Scripts to automate / enforce certain actions
- Triggered when certain (pre-defined) events occur
- Stored in `.git/hooks`
- Named after the event they are associated with (e.g., `pre-commit`, `post-merge`, etc.)
- Can be used for
 - enforcing coding standards
 - preventing accidental commits of sensitive data
 - triggering automatic tests
 - updating documentation
- Samples already present!

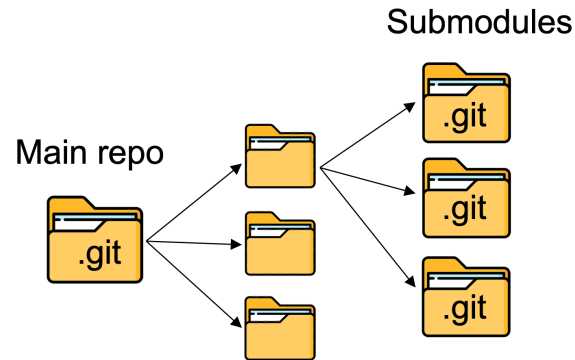
Git Workflow - Exercises

- Exercise 6: `git cherry-pick`
- Exercise 7: Custom Git Hooks

Part 3:

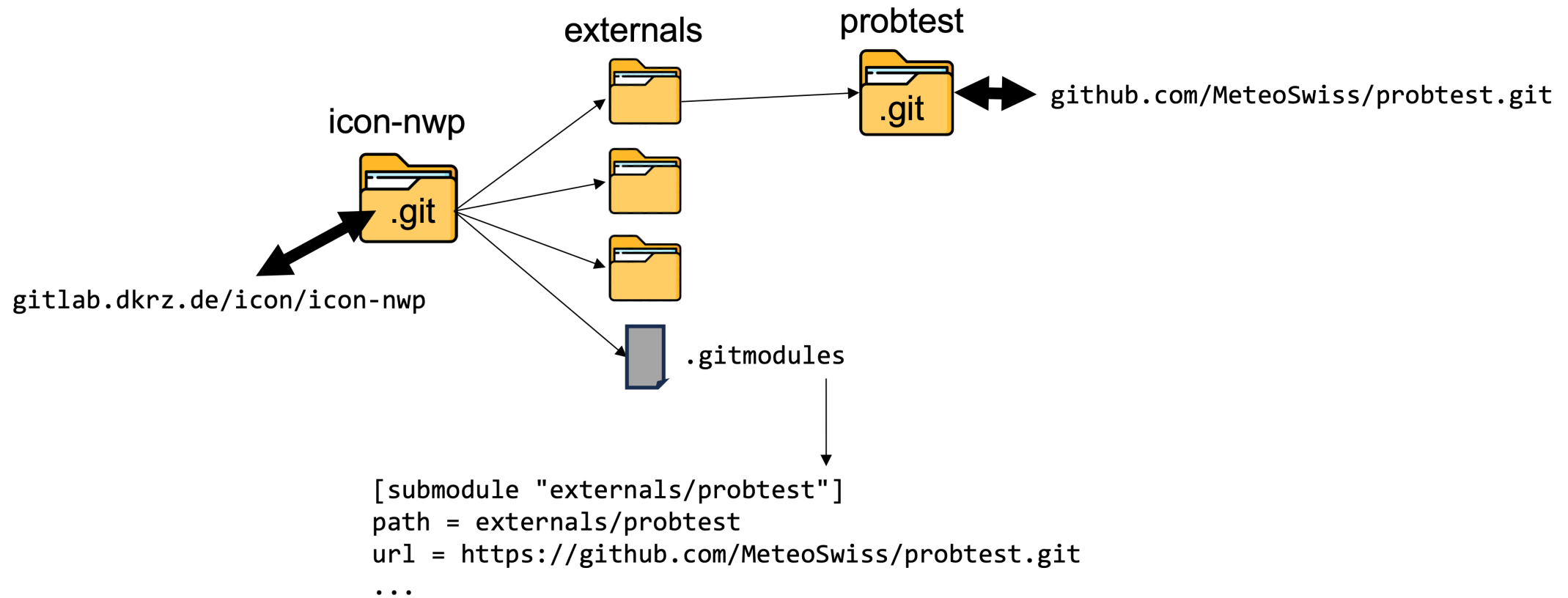
Nesting Git Repositories

Nested Repositories



- Why use it?
 - **Modularity**: Break project into smaller, manageable pieces.
 - **Version Control**: Each submodule has its own Git history and version tracking
 - **Collaboration**: Multiple teams can work on submodules independently
- Options:
 - **Git Submodules**: Git's built-in mechanism
 - **Git Subtrees**: Alternative approach

Parent repository stores reference to external repositories



Cloning a Repository with Submodules

```
git clone
```

By default, Git does NOT clone contents of submodules

```
git clone --recurse-submodules
```

Check out contents of any submodules when cloning parent repo

```
git submodule update --init
```

Get contents of submodules after cloning

Git Submodules



Jeremy Kahn

@jeremyckahn

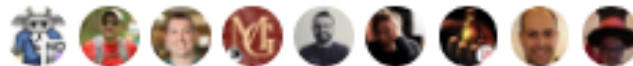
Follow



You there. You're thinking about using a Git submodule. DON'T. Just don't. It's not worth it, ever.

12:19 PM - 16 Dec 2012

16 Retweets 7 Likes



5



16



7



Nesting Git Repositories – Exercises

- Exercise 8: `git submodule`

Part 4:

Useful Tools and Resources

Wide Range of Available Tools

Text editor plugins:

- [magit](#) (Emacs): Wrapper for git commands
- [vim-gitgutter](#) (vim): Improved git diff viewing

Integrated development environment (IDE) integration:

- [Visual Studio Code](#)
- [RStudio](#)
- [Eclipse](#)

Official Git tools:

- [git-gui](#): Focuses on commit generation
- [gitk](#): Focuses on displaying diffs

Terminal prompt changer

- [fancy-git](#)

Git GUIs:

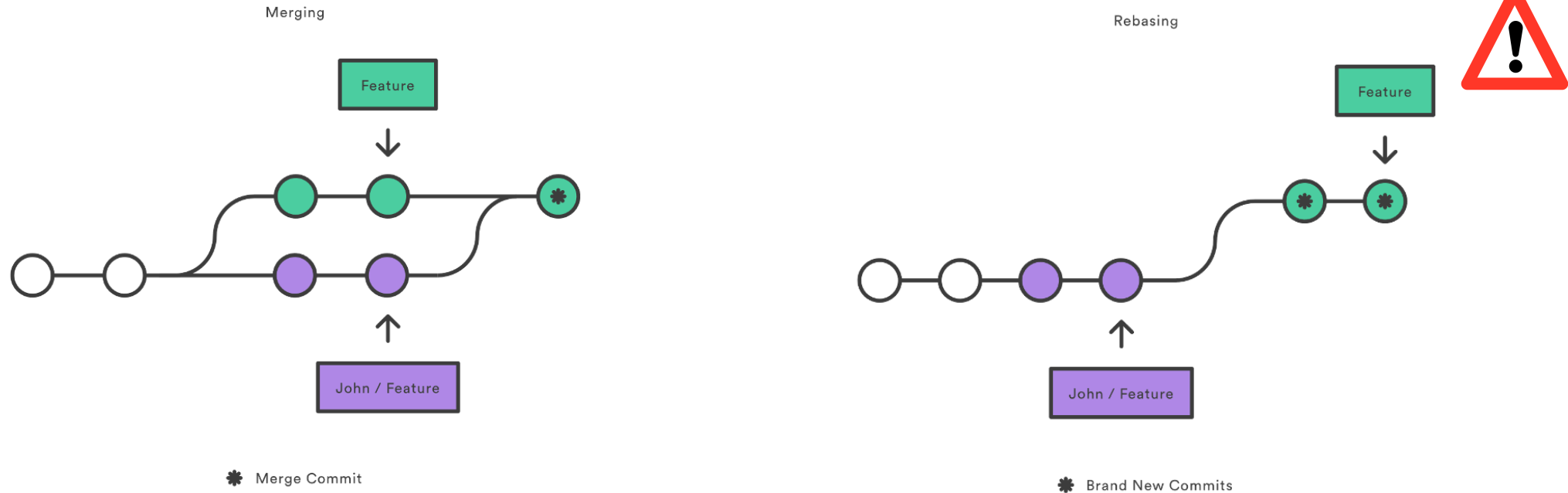
- [Git for Windows](#): Git BASH command line
- [TortoiseGit](#): Windows Shell Interface to Git

Git Resources

- Official Git manual: <http://git-scm.com/>
- GitHub manual: <https://docs.github.com>
- Cheat sheet with useful GitHub commands for quick reference:
<https://education.github.com/git-cheat-sheet-education.pdf>

Bonus Part

git rebase: Alternative to git merge



Source: <https://dzone.com/articles/merging-vs-rebasing>

- Commits are replayed in a different order
- Advantage: keep cleaner commit history
- Should NEVER be used in a shared branch

Questions? Comments?